Assignment 2 Q3(a): Algorithm comparison

Completed matched destinations: 3/3. Requested algorithms: cubic, reno, bbr. Repetitions per algorithm: 3. Duration: 60 s. Sampling: 1.0 s. Seed: 422. Host kernel: 6.18.33.2-microsoft-standard-WSL2.

Test order: seeded initial shuffle; rotate by repetition and candidate index. Each test and its status are recorded in manifest.json. Destinations were shuffled once, then retained only when every algorithm and repetition succeeded. The first complete destination is the representative, chosen without comparing throughput. Failed candidates remain in the manifest and successful partial measurements remain on disk.

The per-run summary gives minimum, median, arithmetic mean, and P95 sampled goodput, duration-weighted goodput, mean/P95 RTT, and retransmitted segments. P95 uses linear interpolation at index (n - 1) * 0.95 in the sorted samples. Aggregate throughput is the mean of duration-weighted per-run goodputs; variability is sample standard deviation across runs. RTT aggregates average the sampled smoothed RTT, not independent packet RTT observations.

dal.speedtest.clouvider.net:5200: highest measured mean goodput: reno (281.301 Mbit/s). Lowest measured mean RTT: bbr (64.530 ms).

- bbr: mean goodput 269.791 Mbit/s, run standard deviation 11.688, range 256.316-277.171.
- cubic: mean goodput 273.962 Mbit/s, run standard deviation 5.633, range 268.468-279.725.
- reno: mean goodput 281.301 Mbit/s, run standard deviation 2.756, range 279.145-284.405.

speed.netfiber.net.il:5201: highest measured mean goodput: reno (119.066 Mbit/s). Lowest measured mean RTT: bbr (177.386 ms).

- reno: mean goodput 119.066 Mbit/s, run standard deviation 0.893, range 118.039-119.667.
- bbr: mean goodput 106.588 Mbit/s, run standard deviation 17.365, range 86.572-117.631.
- cubic: mean goodput 118.085 Mbit/s, run standard deviation 4.160, range 113.306-120.890.

Assignment 2 Q3(a): Algorithm comparison

96.45.41.167:30001: highest measured mean goodput: reno (163.523 Mbit/s). Lowest measured mean RTT: cubic (132.581 ms).

cubic: mean goodput 163.117 Mbit/s, run standard deviation 17.505, range 143.268-176.351.

reno: mean goodput 163.523 Mbit/s, run standard deviation 11.802, range 152.883-176.217.

bbr: mean goodput 147.472 Mbit/s, run standard deviation 6.018, range 141.150-153.131.

Per-run statistics (goodput in Mbit/s; RTT in ms):

dal.speedtest.clouvider.net:5200 bbr run 1: goodput min/median/mean/P95 = 202.814 / 260.731 / 256.315 / 281.472; mean RTT = 62.459; retransmitted segments = 104; matched destination = True.

Window/RTT context: minimum filtered RTT 28.965 ms; median cwnd 2636.0 segments (3816928 bytes); observed-rate BDP proxy 928023 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 cubic run 1: goodput min/median/mean/P95 = 208.026 / 275.568 / 273.692 / 287.345; mean RTT = 66.713; retransmitted segments = 121; matched destination = True.

Window/RTT context: minimum filtered RTT 30.309 ms; median cwnd 1886.0 segments (2730928 bytes); observed-rate BDP proxy 1036914 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 reno run 1: goodput min/median/mean/P95 = 219.239 / 281.231 / 279.146 / 293.892; mean RTT = 73.369; retransmitted segments = 345; matched destination = True.

Window/RTT context: minimum filtered RTT 31.328 ms; median cwnd 1982.0 segments (2869936 bytes); observed-rate BDP proxy 1093132 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 cubic run 2: goodput min/median/mean/P95 = 222.768 / 281.348 / 279.725 / 292.683; mean RTT = 75.246; retransmitted segments = 36; matched destination = True.

Assignment 2 Q3(a): Algorithm comparison

Window/RTT context: minimum filtered RTT 30.801 ms; median cwnd 2061.0 segments (2984328 bytes); observed-rate BDP proxy 1076977 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 reno run 2: goodput min/median/mean/P95 = 220.849 / 285.472 / 284.406 / 296.555; mean RTT = 73.469; retransmitted segments = 63; matched destination = True.

Window/RTT context: minimum filtered RTT 31.606 ms; median cwnd 4657.0 segments (6743336 bytes); observed-rate BDP proxy 1123614 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 bbr run 2: goodput min/median/mean/P95 = 197.247 / 282.652 / 277.170 / 295.825; mean RTT = 71.937; retransmitted segments = 17; matched destination = True.

Window/RTT context: minimum filtered RTT 29.064 ms; median cwnd 3533.0 segments (5115784 bytes); observed-rate BDP proxy 1006962 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 reno run 3: goodput min/median/mean/P95 = 217.907 / 283.042 / 280.354 / 297.142; mean RTT = 74.125; retransmitted segments = 36; matched destination = True.

Window/RTT context: minimum filtered RTT 31.095 ms; median cwnd 7854.0 segments (11372592 bytes); observed-rate BDP proxy 1089693 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 bbr run 3: goodput min/median/mean/P95 = 209.804 / 281.981 / 275.889 / 297.117; mean RTT = 59.194; retransmitted segments = 75; matched destination = True.

Window/RTT context: minimum filtered RTT 29.426 ms; median cwnd 2598.0 segments (3761904 bytes); observed-rate BDP proxy 1014780 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

dal.speedtest.clouvider.net:5200 cubic run 3: goodput min/median/mean/P95 = 17.659 / 277.087 /

Assignment 2 Q3(a): Algorithm comparison

268.470 / 292.971; mean RTT = 64.241; retransmitted segments = 67; matched destination = True.

Window/RTT context: minimum filtered RTT 29.714 ms; median cwnd 2006.0 segments (2904688 bytes); observed-rate BDP proxy 997158 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

a110.speedtest.wobcom.de:5201 bbr run 2: goodput min/median/mean/P95 = 13.759 / 173.237 / 163.875 / 178.361; mean RTT = 126.951; retransmitted segments = 3; matched destination = False.

Window/RTT context: minimum filtered RTT 119.372 ms; median cwnd 5741.0 segments (8312968 bytes); observed-rate BDP proxy 2445246 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

a110.speedtest.wobcom.de:5201 cubic run 2: goodput min/median/mean/P95 = 14.702 / 174.769 / 170.379 / 176.238; mean RTT = 127.200; retransmitted segments = 3; matched destination = False.

Window/RTT context: minimum filtered RTT 120.371 ms; median cwnd 2566.0 segments (3715568 bytes); observed-rate BDP proxy 2563566 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 bbr run 1: goodput min/median/mean/P95 = 59.552 / 251.440 / 212.975 / 285.891; mean RTT = 93.484; retransmitted segments = 26; matched destination = False.

Window/RTT context: minimum filtered RTT 25.500 ms; median cwnd 2733.0 segments (3957384 bytes); observed-rate BDP proxy 678865 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 cubic run 1: goodput min/median/mean/P95 = 17.736 / 188.765 / 183.977 / 256.022; mean RTT = 107.207; retransmitted segments = 311; matched destination = False.

Window/RTT context: minimum filtered RTT 25.948 ms; median cwnd 1870.5 segments (2708484 bytes); observed-rate BDP proxy 596719 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

Assignment 2 Q3(a): Algorithm comparison

speedtest.wdc2.us.leaseweb.net:5201 reno run 1: goodput min/median/mean/P95 = 72.866 / 222.657 / 207.479 / 269.479; mean RTT = 102.964; retransmitted segments = 69; matched destination = False.

Window/RTT context: minimum filtered RTT 29.514 ms; median cwnd 9199.0 segments (13320152 bytes); observed-rate BDP proxy 765435 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 cubic run 2: goodput min/median/mean/P95 = 141.245 / 250.684 / 245.309 / 275.132; mean RTT = 81.463; retransmitted segments = 59; matched destination = False.

Window/RTT context: minimum filtered RTT 26.539 ms; median cwnd 2031.0 segments (2940888 bytes); observed-rate BDP proxy 813787 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 bbr run 2: goodput min/median/mean/P95 = 46.729 / 246.086 / 223.961 / 280.504; mean RTT = 79.575; retransmitted segments = 216; matched destination = False.

Window/RTT context: minimum filtered RTT 25.950 ms; median cwnd 2370.0 segments (3431760 bytes); observed-rate BDP proxy 726474 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 reno run 3: goodput min/median/mean/P95 = 201.369 / 269.235 / 263.684 / 286.277; mean RTT = 78.320; retransmitted segments = 6; matched destination = False.

Window/RTT context: minimum filtered RTT 26.971 ms; median cwnd 5847.0 segments (8466456 bytes); observed-rate BDP proxy 888971 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speedtest.wdc2.us.leaseweb.net:5201 bbr run 3: goodput min/median/mean/P95 = 63.063 / 274.409 / 251.208 / 288.431; mean RTT = 85.323; retransmitted segments = 12; matched destination = False.

Window/RTT context: minimum filtered RTT 25.407 ms; median cwnd 2843.0 segments (4116664 bytes); observed-rate BDP proxy 797801 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

Assignment 2 Q3(a): Algorithm comparison

speedtest.wdc2.us.leaseweb.net:5201 cubic run 3: goodput min/median/mean/P95 = 203.022 / 268.101 / 262.218 / 279.950; mean RTT = 70.258; retransmitted segments = 8; matched destination = False.

Window/RTT context: minimum filtered RTT 27.164 ms; median cwnd 1999.0 segments (2894552 bytes); observed-rate BDP proxy 890363 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

nl.speed.vps1.net:5201 cubic run 1: goodput min/median/mean/P95 = 34.791 / 192.631 / 183.786 / 200.219; mean RTT = 118.500; retransmitted segments = 5; matched destination = False.

Window/RTT context: minimum filtered RTT 104.721 ms; median cwnd 5277.0 segments (7641096 bytes); observed-rate BDP proxy 2405788 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

nl.speed.vps1.net:5201 reno run 1: goodput min/median/mean/P95 = 29.533 / 187.833 / 183.824 / 195.101; mean RTT = 117.804; retransmitted segments = 15; matched destination = False.

Window/RTT context: minimum filtered RTT 107.944 ms; median cwnd 5692.0 segments (8242016 bytes); observed-rate BDP proxy 2480309 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

nl.speed.vps1.net:5201 bbr run 1: goodput min/median/mean/P95 = 18.260 / 189.426 / 180.351 / 195.177; mean RTT = 116.421; retransmitted segments = 43; matched destination = False.

Window/RTT context: minimum filtered RTT 107.349 ms; median cwnd 5754.0 segments (8331792 bytes); observed-rate BDP proxy 2420024 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

nl.speed.vps1.net:5201 cubic run 2: goodput min/median/mean/P95 = 29.511 / 190.652 / 185.730 / 194.965; mean RTT = 116.274; retransmitted segments = 3; matched destination = False.

Window/RTT context: minimum filtered RTT 108.469 ms; median cwnd 5954.0 segments (8621392 bytes); observed-rate BDP proxy 2518210 bytes. This proxy uses achieved goodput, so it cannot

Assignment 2 Q3(a): Algorithm comparison

independently identify bottleneck capacity.

nl.speed.vps1.net:5201 cubic run 3: goodput min/median/mean/P95 = 29.523 / 192.030 / 187.404 / 196.124; mean RTT = 116.371; retransmitted segments = 6; matched destination = False.

Window/RTT context: minimum filtered RTT 107.013 ms; median cwnd 5914.0 segments (8563472 bytes); observed-rate BDP proxy 2506805 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 reno run 1: goodput min/median/mean/P95 = 3.588 / 122.907 / 118.041 / 127.958; mean RTT = 185.027; retransmitted segments = 12; matched destination = True.

Window/RTT context: minimum filtered RTT 171.392 ms; median cwnd 5952.0 segments (8618496 bytes); observed-rate BDP proxy 2528875 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 bbr run 1: goodput min/median/mean/P95 = 2.512 / 120.574 / 115.562 / 128.968; mean RTT = 178.686; retransmitted segments = 2; matched destination = True.

Window/RTT context: minimum filtered RTT 173.182 ms; median cwnd 5720.0 segments (8282560 bytes); observed-rate BDP proxy 2501635 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 cubic run 1: goodput min/median/mean/P95 = 3.588 / 122.346 / 120.061 / 127.930; mean RTT = 179.203; retransmitted segments = 17; matched destination = True.

Window/RTT context: minimum filtered RTT 173.757 ms; median cwnd 5942.0 segments (8604016 bytes); observed-rate BDP proxy 2607640 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 bbr run 2: goodput min/median/mean/P95 = 2.465 / 123.697 / 117.634 / 129.107; mean RTT = 176.741; retransmitted segments = 0; matched destination = True.

Window/RTT context: minimum filtered RTT 171.170 ms; median cwnd 5716.0 segments (8276768 bytes);

Assignment 2 Q3(a): Algorithm comparison

observed-rate BDP proxy 2516855 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 cubic run 2: goodput min/median/mean/P95 = 3.586 / 117.321 / 113.307 / 123.802; mean RTT = 187.737; retransmitted segments = 2; matched destination = True.

Window/RTT context: minimum filtered RTT 173.293 ms; median cwnd 2622.0 segments (3796656 bytes); observed-rate BDP proxy 2454387 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 reno run 2: goodput min/median/mean/P95 = 3.590 / 122.629 / 119.492 / 126.308; mean RTT = 180.535; retransmitted segments = 20; matched destination = True.

Window/RTT context: minimum filtered RTT 173.809 ms; median cwnd 5820.0 segments (8427360 bytes); observed-rate BDP proxy 2596090 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 cubic run 3: goodput min/median/mean/P95 = 3.589 / 123.445 / 120.891 / 128.412; mean RTT = 178.770; retransmitted segments = 4; matched destination = True.

Window/RTT context: minimum filtered RTT 170.633 ms; median cwnd 3964.0 segments (5739872 bytes); observed-rate BDP proxy 2578467 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 reno run 3: goodput min/median/mean/P95 = 3.589 / 122.837 / 119.667 / 127.144; mean RTT = 179.453; retransmitted segments = 9; matched destination = True.

Window/RTT context: minimum filtered RTT 173.343 ms; median cwnd 5740.0 segments (8311520 bytes); observed-rate BDP proxy 2592920 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

speed.netfiber.net.il:5201 bbr run 3: goodput min/median/mean/P95 = 2.176 / 90.942 / 86.574 / 120.681; mean RTT = 176.731; retransmitted segments = 396; matched destination = True.

Assignment 2 Q3(a): Algorithm comparison

Window/RTT context: minimum filtered RTT 172.429 ms; median cwnd 4060.0 segments (5878880 bytes); observed-rate BDP proxy 1865930 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 reno run 1: goodput min/median/mean/P95 = 3.607 / 135.962 / 132.816 / 139.885; mean RTT = 162.959; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 154.782 ms; median cwnd 4045.0 segments (5760080 bytes); observed-rate BDP proxy 2569662 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 cubic run 2: goodput min/median/mean/P95 = 7.171 / 135.784 / 132.816 / 139.378; mean RTT = 162.767; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 155.357 ms; median cwnd 4029.0 segments (5737296 bytes); observed-rate BDP proxy 2579196 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 reno run 2: goodput min/median/mean/P95 = 6.435 / 132.785 / 129.360 / 135.174; mean RTT = 166.385; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 159.340 ms; median cwnd 4036.0 segments (5747264 bytes); observed-rate BDP proxy 2576474 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 bbr run 2: goodput min/median/mean/P95 = 2.921 / 127.339 / 120.666 / 131.112; mean RTT = 172.105; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 166.093 ms; median cwnd 5853.0 segments (8334672 bytes); observed-rate BDP proxy 2505220 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 reno run 3: goodput min/median/mean/P95 = 3.254 / 124.793 / 122.265 / 139.106; mean RTT = 176.513; retransmitted segments = 0; matched destination = False.

Assignment 2 Q3(a): Algorithm comparison

Window/RTT context: minimum filtered RTT 153.796 ms; median cwnd 4030.0 segments (5738720 bytes); observed-rate BDP proxy 2350437 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 bbr run 3: goodput min/median/mean/P95 = 2.853 / 126.108 / 120.430 / 131.095; mean RTT = 172.668; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 166.846 ms; median cwnd 5788.0 segments (8242112 bytes); observed-rate BDP proxy 2511609 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

iperf.pendc.com:5201 cubic run 3: goodput min/median/mean/P95 = 3.050 / 131.528 / 110.381 / 135.569; mean RTT = 184.828; retransmitted segments = 0; matched destination = False.

Window/RTT context: minimum filtered RTT 158.601 ms; median cwnd 2049.0 segments (2917776 bytes); observed-rate BDP proxy 2188278 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 cubic run 1: goodput min/median/mean/P95 = 23.359 / 174.920 / 169.732 / 179.385; mean RTT = 127.934; retransmitted segments = 35; matched destination = True.

Window/RTT context: minimum filtered RTT 117.895 ms; median cwnd 3970.0 segments (5748560 bytes); observed-rate BDP proxy 2501320 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 reno run 1: goodput min/median/mean/P95 = 24.358 / 180.366 / 176.219 / 185.585; mean RTT = 121.692; retransmitted segments = 3; matched destination = True.

Window/RTT context: minimum filtered RTT 114.704 ms; median cwnd 4166.0 segments (6032368 bytes); observed-rate BDP proxy 2526598 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 bbr run 1: goodput min/median/mean/P95 = 0.000 / 176.741 / 148.138 / 186.682;

Assignment 2 Q3(a): Algorithm comparison

mean RTT = 192.792; retransmitted segments = 435; matched destination = True.

Window/RTT context: minimum filtered RTT 112.199 ms; median cwnd 5749.0 segments (8324552 bytes); observed-rate BDP proxy 2077564 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 reno run 2: goodput min/median/mean/P95 = 16.486 / 159.570 / 152.885 / 183.652; mean RTT = 141.376; retransmitted segments = 6; matched destination = True.

Window/RTT context: minimum filtered RTT 114.346 ms; median cwnd 3974.0 segments (5754352 bytes); observed-rate BDP proxy 2185195 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 bbr run 2: goodput min/median/mean/P95 = 12.026 / 164.477 / 153.135 / 180.577; mean RTT = 141.953; retransmitted segments = 7; matched destination = True.

Window/RTT context: minimum filtered RTT 109.866 ms; median cwnd 5780.0 segments (8369440 bytes); observed-rate BDP proxy 2102985 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 cubic run 2: goodput min/median/mean/P95 = 12.570 / 156.311 / 143.270 / 170.208; mean RTT = 147.650; retransmitted segments = 0; matched destination = True.

Window/RTT context: minimum filtered RTT 119.790 ms; median cwnd 2030.0 segments (2939440 bytes); observed-rate BDP proxy 2145256 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 bbr run 3: goodput min/median/mean/P95 = 2.244 / 183.417 / 141.152 / 190.987; mean RTT = 117.061; retransmitted segments = 99; matched destination = True.

Window/RTT context: minimum filtered RTT 108.886 ms; median cwnd 4032.0 segments (5838336 bytes); observed-rate BDP proxy 1921160 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

Assignment 2 Q3(a): Algorithm comparison

96.45.41.167:30001 cubic run 3: goodput min/median/mean/P95 = 29.351 / 186.270 / 176.354 / 193.780; mean RTT = 122.158; retransmitted segments = 85; matched destination = True.

Window/RTT context: minimum filtered RTT 109.457 ms; median cwnd 3965.0 segments (5741320 bytes); observed-rate BDP proxy 2412853 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

96.45.41.167:30001 reno run 3: goodput min/median/mean/P95 = 20.127 / 178.491 / 161.471 / 186.671; mean RTT = 147.005; retransmitted segments = 2; matched destination = True.

Window/RTT context: minimum filtered RTT 112.252 ms; median cwnd 3984.0 segments (5768832 bytes); observed-rate BDP proxy 2265662 bytes. This proxy uses achieved goodput, so it cannot independently identify bottleneck capacity.

Interpretation: cwnd limits in-flight segments; its byte budget is cwnd * MSS. Without another bottleneck, the window-limited rate is approximately cwnd * MSS / RTT. The bandwidth-delay product estimates bytes needed in flight at bottleneck bandwidth and base RTT. Extra in-flight bytes can accumulate in queues. RTT growth at similar goodput is consistent with extra queueing; a larger cwnd alone does not establish higher useful throughput.

Reno increases its congestion window additively during congestion avoidance and reduces it after loss. CUBIC probes with a cubic window trajectory and can recover capacity faster on large bandwidth-delay paths. BBR estimates delivery bandwidth and propagation RTT and uses a model with pacing. These are expected mechanisms, not evidence that a particular algorithm won on an unmeasured network regime. The installed BBR version depends on the host kernel.

Retransmissions are a loss proxy, not a direct packet-loss probability. Reordering and spurious retransmissions can contribute. Window reductions may precede a later goodput drop; simultaneous scatter plots do not capture every delayed response. Public-server rate caps, receiver windows, CPU limits, and application limits can hide congestion-control differences.

Limitations: sequential tests share destinations and settings but cannot hold Internet cross-traffic or routes constant. Rotating order reduces systematic order bias without eliminating it. Short runs include startup. Excluding failed candidate groups can bias the sample toward reliable

Assignment 2 Q3(a): Algorithm comparison

servers. Differences of the same magnitude as run variability are not strong evidence of an advantage. Establishing operating regimes causally requires controlled bottleneck bandwidth, RTT, queue size, and loss; public traces alone are insufficient.

AI assistance: an AI coding assistant helped implement experiment automation, plotting, and draft the algorithm design and explanatory text. The team must review the code, measurements, and claims and be able to explain them during evaluation.

Per-run goodput summary (Mbit/s); * = incomplete destination group

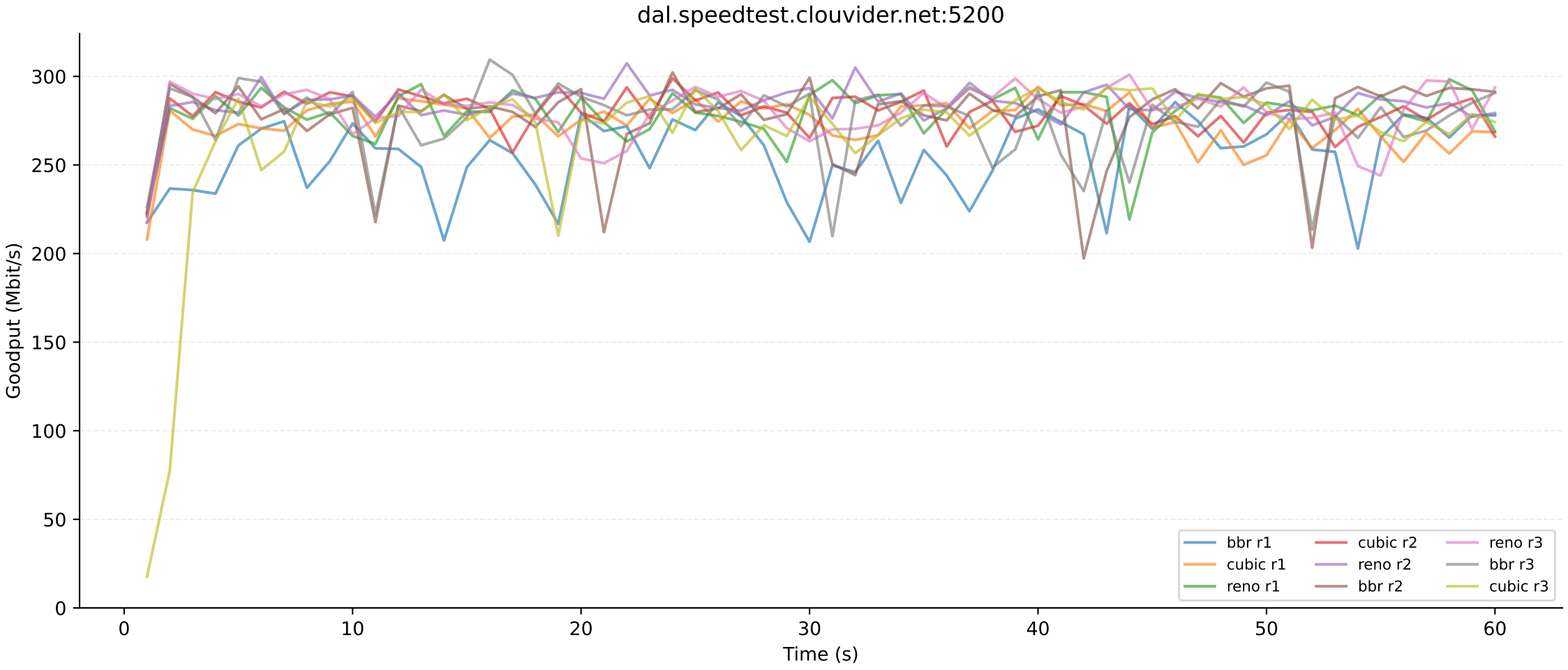
Destination	Algorithm	Run	Min	Median	Mean	P95
dal.speedtest.clouvider.net:5200	bbr	1	202.814	260.731	256.315	281.472
dal.speedtest.clouvider.net:5200	cubic	1	208.026	275.568	273.692	287.345
dal.speedtest.clouvider.net:5200	reno	1	219.239	281.231	279.146	293.892
dal.speedtest.clouvider.net:5200	cubic	2	222.768	281.348	279.725	292.683
dal.speedtest.clouvider.net:5200	reno	2	220.849	285.472	284.406	296.555
dal.speedtest.clouvider.net:5200	bbr	2	197.247	282.652	277.170	295.825
dal.speedtest.clouvider.net:5200	reno	3	217.907	283.042	280.354	297.142
dal.speedtest.clouvider.net:5200	bbr	3	209.804	281.981	275.889	297.117
dal.speedtest.clouvider.net:5200	cubic	3	17.659	277.087	268.470	292.971
a110.speedtest.wobcom.de:5201*	bbr	2	13.759	173.237	163.875	178.361
a110.speedtest.wobcom.de:5201*	cubic	2	14.702	174.769	170.379	176.238
speedtest.wdc2.us.leaseweb.net:5201*	bbr	1	59.552	251.440	212.975	285.891
speedtest.wdc2.us.leaseweb.net:5201*	cubic	1	17.736	188.765	183.977	256.022
speedtest.wdc2.us.leaseweb.net:5201*	reno	1	72.866	222.657	207.479	269.479
speedtest.wdc2.us.leaseweb.net:5201*	cubic	2	141.245	250.684	245.309	275.132
speedtest.wdc2.us.leaseweb.net:5201*	bbr	2	46.729	246.086	223.961	280.504
speedtest.wdc2.us.leaseweb.net:5201*	reno	3	201.369	269.235	263.684	286.277
speedtest.wdc2.us.leaseweb.net:5201*	bbr	3	63.063	274.409	251.208	288.431

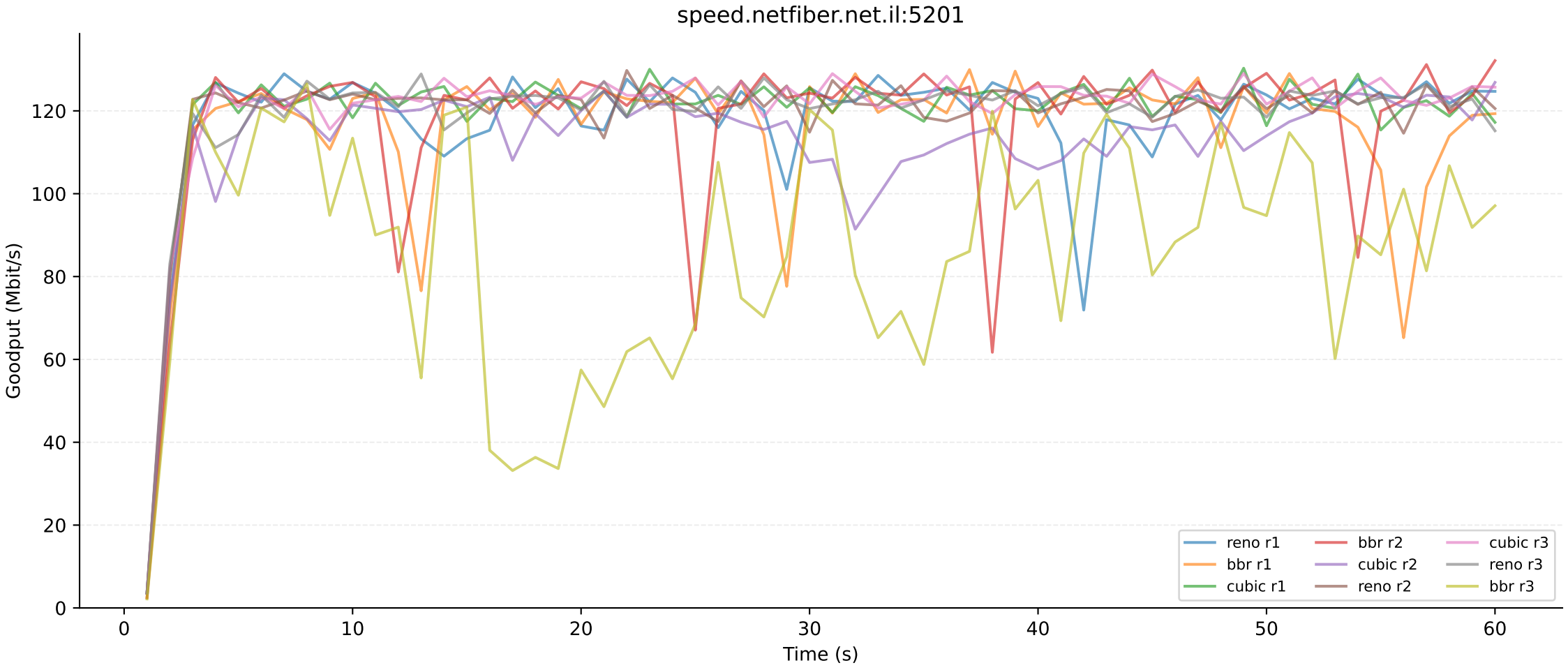
Per-run goodput summary (Mbit/s); * = incomplete destination group

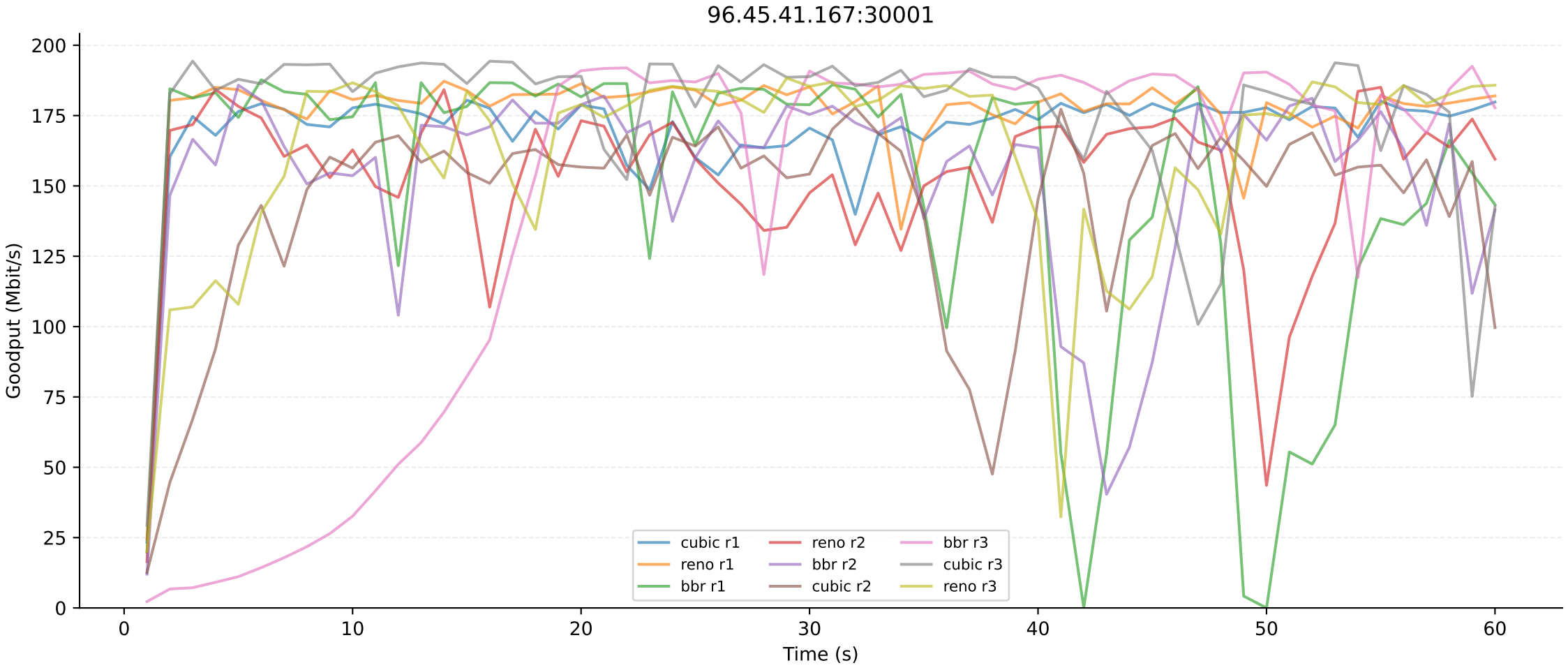
Destination	Algorithm	Run	Min	Median	Mean	P95
speedtest.wdc2.us.leaseweb.net:5201*	cubic	3	203.022	268.101	262.218	279.950
nl.speed.vps1.net:5201*	cubic	1	34.791	192.631	183.786	200.219
nl.speed.vps1.net:5201*	reno	1	29.533	187.833	183.824	195.101
nl.speed.vps1.net:5201*	bbr	1	18.260	189.426	180.351	195.177
nl.speed.vps1.net:5201*	cubic	2	29.511	190.652	185.730	194.965
nl.speed.vps1.net:5201*	cubic	3	29.523	192.030	187.404	196.124
speed.netfiber.net.il:5201	reno	1	3.588	122.907	118.041	127.958
speed.netfiber.net.il:5201	bbr	1	2.512	120.574	115.562	128.968
speed.netfiber.net.il:5201	cubic	1	3.588	122.346	120.061	127.930
speed.netfiber.net.il:5201	bbr	2	2.465	123.697	117.634	129.107
speed.netfiber.net.il:5201	cubic	2	3.586	117.321	113.307	123.802
speed.netfiber.net.il:5201	reno	2	3.590	122.629	119.492	126.308
speed.netfiber.net.il:5201	cubic	3	3.589	123.445	120.891	128.412
speed.netfiber.net.il:5201	reno	3	3.589	122.837	119.667	127.144
speed.netfiber.net.il:5201	bbr	3	2.176	90.942	86.574	120.681
iperf.pendc.com:5201*	reno	1	3.607	135.962	132.816	139.885
iperf.pendc.com:5201*	cubic	2	7.171	135.784	132.816	139.378
iperf.pendc.com:5201*	reno	2	6.435	132.785	129.360	135.174

Per-run goodput summary (Mbit/s); * = incomplete destination group

Destination	Algorithm	Run	Min	Median	Mean	P95
iperf.pendc.com:5201*	bbr	2	2.921	127.339	120.666	131.112
iperf.pendc.com:5201*	reno	3	3.254	124.793	122.265	139.106
iperf.pendc.com:5201*	bbr	3	2.853	126.108	120.430	131.095
iperf.pendc.com:5201*	cubic	3	3.050	131.528	110.381	135.569
96.45.41.167:30001	cubic	1	23.359	174.920	169.732	179.385
96.45.41.167:30001	reno	1	24.358	180.366	176.219	185.585
96.45.41.167:30001	bbr	1	0.000	176.741	148.138	186.682
96.45.41.167:30001	reno	2	16.486	159.570	152.885	183.652
96.45.41.167:30001	bbr	2	12.026	164.477	153.135	180.577
96.45.41.167:30001	cubic	2	12.570	156.311	143.270	170.208
96.45.41.167:30001	bbr	3	2.244	183.417	141.152	190.987
96.45.41.167:30001	cubic	3	29.351	186.270	176.354	193.780
96.45.41.167:30001	reno	3	20.127	178.491	161.471	186.671


dal.speedtest.clouvider.net:5200
Goodput (Mbit/s)
Time (s)
0
50
100
150
200
250
300
0
10
20
30
40
50
60
bbr r1
cubic r1
reno r1
cubic r2
reno r2
bbr r2
reno r3
bbr r3
cubic r3


speed.netfiber.net.il:5201
Goodput (Mbit/s)
Time (s)
0
20
40
60
80
100
120
0
10
20
30
40
50
60
reno r1
bbr r1
cubic r1
bbr r2
cubic r2
reno r2
cubic r3
reno r3
bbr r3


96.45.41.167:30001
Goodput (Mbit/s)
Time (s)
0
25
50
75
100
125
150
175
200
0
10
20
30
40
50
60
cubic r1
reno r1
bbr r1
reno r2
bbr r2
cubic r2
bbr r3
cubic r3
reno r3

dal.speedtest.clouvider.net:5200 | matched settings; individual repetitions

cubic | reno | bbr

Congestion window (segments)

RTT (ms)

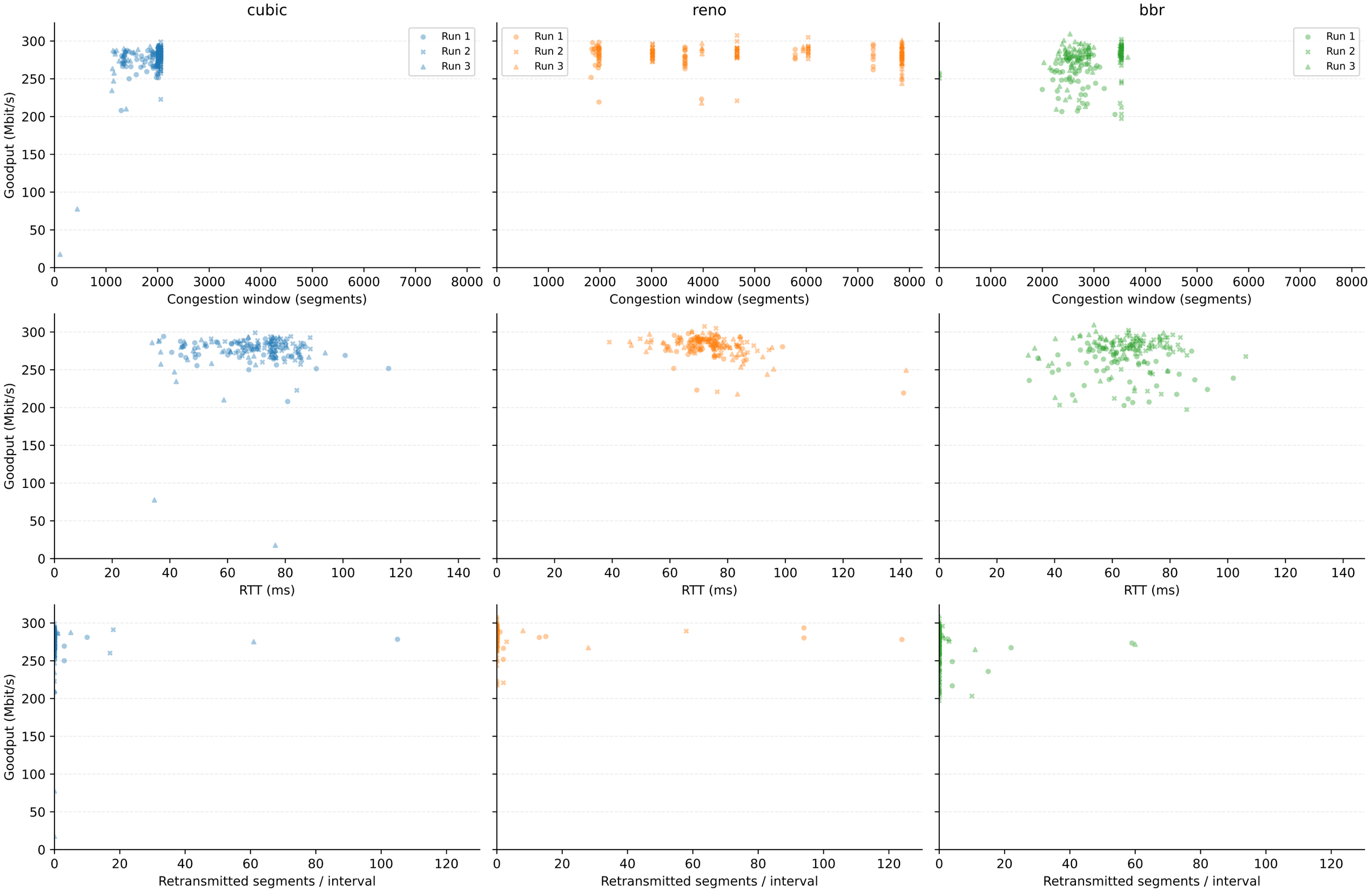
Retransmitted segments / interval

Goodput (Mbit/s)

Time (s)

Run 1
Run 2
Run 3

dal.speedtest.clouvider.net:5200 | all repetitions; common scales per metric



Assignment 2 Q3(b): Algorithm design

Q3(b): RTT, throughput, and loss based window control

Status: proposed design. The completed q3-test dataset compares CUBIC, Reno, and BBR at three destinations with three 60-second repetitions each. Those measurements informed the design rationale below; the proposed algorithm itself has not been implemented or measured. Assignment 3 must implement and evaluate it separately.

State and measurements

Maintain cwnd and ssthresh in segments, initialized to 10 and infinity; obey any lower kernel or peer limit. Minimum cwnd is 2. Let MSS be the current sending maximum segment size in bytes. Use the host TCP stack for ACK processing, retransmission, recovery, and timeout handling.

Update once per measured RTT, using only newly acknowledged data. During each epoch collect acknowledged bytes, elapsed time, RTT samples, and newly retransmitted segments. Goodput is acknowledged bytes / elapsed seconds, in bytes/s. Estimate throughput with an EWMA: rate = 0.75 * previous_rate + 0.25 * epoch_goodput; initialize it from the first valid epoch.

Let base_rtt be the minimum valid RTT in a rolling 10-second window. Let queue_ratio = max(0, srtt / base_rtt - 1). Keep a rolling maximum of non-application-limited epoch goodput over 10 seconds as bandwidth_estimate. Then bdp_segments = max(2, bandwidth_estimate * base_rtt / MSS). RTT values in this calculation are seconds. This is an observed delivery-rate estimate, not a measurement of physical link capacity.

Assignment 2 Q3(b): Algorithm design

A loss event means a newly entered TCP recovery episode, not every ACK that reports retransmissions. Only reduce once in that episode, respecting the kernel recovery boundary. Ignore epochs with no newly acknowledged bytes for growth; timeout handling still applies. Freeze delay and throughput growth decisions when the sender is application limited or receiver-window limited.

Pseudocode

State: cwnd = 10; ssthresh = infinity; mode = STARTUP; rate = unset

On valid ACK: accumulate delivered bytes and RTT samples for this epoch

On entering a new loss recovery episode:

    cwnd = max(2, floor(0.70 * cwnd))

    ssthresh = cwnd; mode = AVOIDANCE

    mark recovery boundary; reset epoch accounting

On retransmission timeout:

    ssthresh = max(2, floor(cwnd / 2))

    cwnd = 2; mode = AVOIDANCE; reset epoch accounting

Once per RTT, after ACK progress, outside loss recovery:

    update base_rtt, bandwidth_estimate, and rate

Assignment 2 Q3(b): Algorithm design

```
if application limited or receiver-window limited: hold cwnd; finish epoch
if mode == STARTUP:
    if queue_ratio < 0.10:
        cwnd = cwnd + min(newly_acked_segments, cwnd)
        finish epoch
    mode = AVOIDANCE; ssthresh = cwnd
if queue_ratio > 0.25:
    cwnd = max(2, floor(0.90 * cwnd))
else if queue_ratio < 0.10:
    if cwnd < 1.25 * bdp_segments or rate > 1.02 * previous_rate:
        cwnd = cwnd + 1
    else if at least 8 RTT epochs have passed since the last probe:
        cwnd = cwnd + 1
        reset probe timer
else:
    hold cwnd
enforce kernel cwnd clamp; finish epoch
```

Assignment 2 Q3(b): Algorithm design

Initialize previous_rate from the first epoch and skip the rate-growth comparison for that epoch. Count newly acknowledged segments with fractional credit across ACKs, so delayed ACKs and partial segments do not inflate growth. Maintain the probe timer across epochs, incrementing it once per eligible RTT and resetting it on a probe or reduction. Do not stack a delay reduction on a loss reduction within the same recovery epoch. A kernel/eBPF implementation must let the stack apply timeout behavior exactly once rather than performing a second reduction in an ACK callback.

Design choices

The existing CUBIC traces motivated retaining acknowledged-byte goodput, smoothed RTT, and retransmission counters as the observable inputs. Two 15-second runs to 185.180.12.40:5201 had duration-weighted goodputs of 164.20 and 93.62 Mbit/s, mean smoothed RTTs of 128.18 and 129.08 ms, and retransmission deltas totaling 0 and 147 segments. Their median congestion windows were 4165 and 1135.5 segments. These runs used the same algorithm: they show substantial variability and motivate measuring throughput and loss alongside RTT, rather than inferring useful rate from cwnd alone. They do not establish that retransmissions caused the entire goodput difference, and they do not establish optimal thresholds. The source files are results/20261018T035444Z_185.180.12.40_5201.csv and results/20261018T035817Z_185.180.12.40_5201.csv; calculations use run_statistics in comparison_plots.py. The 10% and 25% queue thresholds create a band in which the sender holds its window. Low delay allows probing; sustained queue growth causes a small reduction before loss. Loss triggers a larger reduction. The eight-RTT probe avoids permanently accepting an underestimated bandwidth estimate after a path gains capacity. The BDP gate limits routine growth when extra window stops increasing useful delivery. Startup provides initial discovery without an established bandwidth estimate.

Assignment 2 Q3(b): Algorithm design

The full baseline comparison reinforces the use of all three signals. In Dallas, Reno's median cwnd rose from 1,982 to 7,854 segments across repetitions while goodput stayed near 279-284 Mbit/s, motivating a throughput gate on routine window growth. On Netfiber, BBR's third run fell to 86.57 Mbit/s with 396 retransmissions despite mean RTT near the path baseline, motivating a separate loss response rather than relying on RTT alone. At 96.45.41.167, one BBR interval had zero acknowledged goodput and a 1,559.64 ms smoothed RTT, motivating a conservative response to sustained delay inflation as well. These are associations in sequential public-path tests, not proof of any mechanism or validation of the proposed thresholds. The analysis and source measurements are in results/q3-test/observations.md and its referenced CSV files.

The constants remain initial hypotheses. We retain the previously shared pseudocode and thresholds so this handoff stays consistent; the new data do not justify presenting those constants as tuned or optimal. Validate them in controlled tests and document any changes in Assignment 3. A rolling base RTT adapts after route changes but can mistake a persistent queue for propagation delay when old samples expire. A rolling bandwidth maximum also lags rapid capacity decreases.

Fairness and stability

Under equal RTTs, persistent backlog, similar packet sizes, and a shared drop-tail bottleneck, equal additive growth with proportional loss reductions can move flows toward similar windows. The queue response can reduce the amplitude of a congestion cycle. This is an expectation, not a proof of fairness or convergence.

Flows with different RTTs update at different frequencies, so RTT fairness is not guaranteed. A delay-sensitive flow may yield to CUBIC or Reno flows that maintain queues. Its estimated minimum

Assignment 2 Q3(b): Algorithm design

RTT may include standing queues, and independent bandwidth estimates can overstate each flow's fair share. The throughput gate can also preserve an unfair initial allocation. Public Internet measurements cannot isolate these effects.

For a single flow on a fixed path, the hold band and rate smoothing should reduce rapid changes, but periodic probing and multiplicative reductions can produce a limit cycle rather than a fixed window. Delayed feedback, ACK compression, reverse-path congestion, random wireless losses, and changing routes can destabilize decisions. Evaluate equal and unequal RTT competing flows, utilization, queue delay, recovery, and Jain's fairness index before claiming stable or fair operation.

Assignment 3 handoff

Implement this state machine in a kernel congestion-control module or eBPF struct_ops program. A Python client cannot implement a kernel congestion window policy by selecting TCP_CONGESTION alone. Use kernel delivery/rate samples and ACK/recovery callbacks to supply the inputs above. Test that callbacks preserve the once-per-RTT and once-per-recovery rules. Record callback mappings, unavailable signals, parameter changes, exact kernel version, and the actual loaded algorithm name. The comparison command accepts extra algorithm names, so it can evaluate the custom algorithm alongside cubic, reno, and bbr with 10-second runs.